Structured Programming Methods

Published by:
NCC Education Limited
The Towers, Towers Business Park
Wilmslow Road, Didsbury
Manchester M20 2EZ

UK

Chap 4: Further Programming Techniques

Procedures and Functions

Introduction

The top down design process essentially means splitting up a problem into its component parts over and over again, until a stage is reached where building a solution becomes much easier.

Procedures and functions are the building blocks which are needed to complete this process. Procedures and functions are sections of a program, or subprograms, which are written to perform a specific task and this will involve operations on data. They are written once but can be used many times during the execution of the program, and each time the procedure or function is used the data can be different.

Definition: procedure

A section of a program that carries out some well-defined operation on data specified by parameters. It can be called from anywhere in a program, and different parameters can be provided for each call. The term procedure is generally used in the context of high level languages; in assembly language the word subroutine is more commonly employed.

Definition: function

A program unit that, given values for input parameters, computes a value. Examples include the standard functions such as sin(x), cos(x), exp(x); in addition most languages permit user-defined functions. A function is a 'black box' that can be used without any knowledge or understanding of the detail of its internal working. In some languages a function may have side effects.

The normal control sequence of the program, where instructions are obeyed in the order in which they were written, is interrupted by a call to the procedure or function. When the instructions within the procedure or function have been completed, control is returned to the instruction which called it and thus the next instruction will be the one after the instruction which called the procedure or function.

The data to be used during the execution of the procedure or function is passed at the time of calling by using variables called parameters. These will be discussed later in this chapter, after the section concerning local and global variables.

Study Note

The difference between a procedure and a function is that a function will return a single value after a call, whereas a procedure will do a particular task but not necessarily return a value. Since the function will return a value, the data type of the function must be declared.

You need to be aware that each structured computer language has its own way of providing these components, e.g. Pascal, which was designed to teach these concepts, has both procedures and functions, whereas C has only functions (which can be used as the equivalent of procedures) and the original BASIC only provided subroutines (which could be used to perform tasks and return values to the calling program).

In our pseudo code we use both procedures and functions. When we want to use a procedure or a function we shall simply give it a name, and call it.

Definition: call

To transfer control to a subroutine or procedure, with provision for return to the instruction following the call at the end of the execution of the subroutine or procedure.

Definition: program unit

A constituent part of a large program, and in some sense self-contained.

Definition: side effects

An effect of a program unit that is not apparent from its parameters, for example altering a non-local variable or performing input/output.

Definition: subroutine

A piece of code which is obeyed ‘out of line’, i.e. control is transferred to the subroutine, and on its completion control reverts to the instruction following the call.

(The instruction code of the CPU usually provides subroutine jump and return instructions to facilitate this operation.) A subroutine saves space since it occurs only once in the program, though it may be called from many different places in the program. It also facilitates the construction of large programs since subroutines can be formed into libraries for general use. (The same concept appears in high level languages as the procedure.)

Definition: top down development

An approach to program development in which progress is made by defining required elements in terms of more basic elements, beginning with the required program and ending when the implementation language has been reached. At every stage during top-down development each of the undefined elements from the previous stage is defined. In order to do this, an appropriate collection of more basic elements is introduced, and the undefined elements are defined in terms of these more basic elements ('more basic' meaning that the element is closer to the level that can be directly expressed in the implementation language). These more basic elements will in turn be defined at the next stage in terms of still more basic elements, and so on until at some stage the elements can be defined directly in the implementation language.

In practice 'pure' top-down development is not possible, the choice of more basic elements at each stage must always be guided by an awareness of the facilities of the implementation language, and even then it will often be discovered at a later stage that some earlier choice was inappropriate, leading to a need for iteration. Compare bottom-up development.

Local and Global Variables

In order to ensure that one procedure or function does not interfere with another, it is very important to keep track of the contents of variables. (This was mentioned earlier in the discussion of the FOR…ENDFOR loop.) The safest and easiest way to keep track of variables is only to use them within individual procedures and not throughout the whole program. This is accomplished by the inclusion of the facility to declare variables as local. The values of local variables are lost as soon as the procedure or function call has concluded.

If procedures use only local variables, they are ‘self contained’, and thus much easier to test and amend.

Global variables keep any values assigned to them and although they can be used anywhere in the program, they can also be influenced or changed by any part of the program. It is for this reason that they should be used with great care!

In our pseudo code, global variables will be listed at the start of the code and local variables within the procedure or function in which they are used.

Definition: local
A term applied to variables that are accessible only in a restricted part of a program, typically in a procedure or function body. By contrast, global variables are accessible throughout a program.

Definition: global
A term used to describe the scope of a variable: global variables are accessible from all parts of a program. By contrast, local variables are accessible only in the program module within which they are defined.

Procedures
To illustrate the use of procedures we will write a program to provide a simple quiz on capital cities, basic mathematics or simple chemistry. This example will illustrate the use of procedures with local variables; the use of parameters will be covered in a later example. The diagram of the algorithm is shown in Figure 4.1.

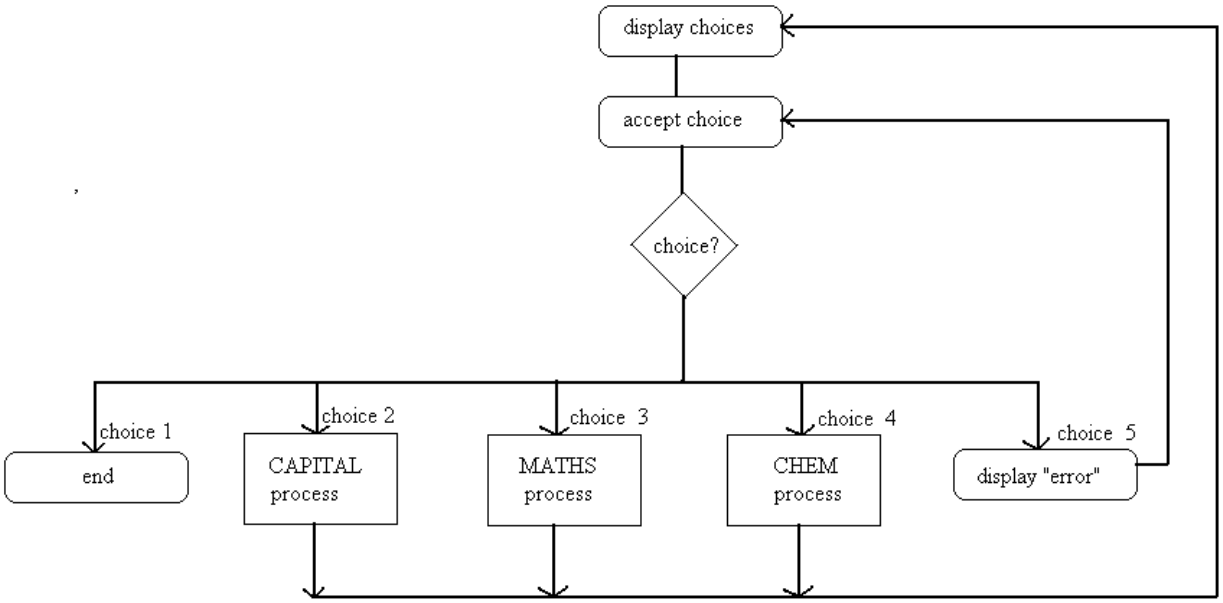


Figure 4.1: Diagram of the Quiz program

The procedure names chosen for this example are capital, maths and chem. The procedures are called from the first block of code; we have called this procedure main. In the pseudo code, all procedures and functions will start with the name and finish with the statement ENDPROCEDURE or ENDFUNCTION.

```
main program
    choice is a local variable OF TYPE Integer
    WHILE choice <> 4 DO
        DISPLAY " Choose which quiz you would like to try"
        DISPLAY " Type in the quiz number"
        DISPLAY " 1. Capital Cities"
        DISPLAY " 2. Basic Maths"
        DISPLAY " 3. Chemical Symbols"
        DISPLAY " 4. End the program"
        REPEAT
            ACCEPT choice
            IF choice < 1 or choice > 4
                THEN DISPLAY " Error : choose again"
```

```
        ENDIF
    UNTIL choice > 0 AND choice <5
    IF choice = 1 THEN Capital
    IF choice = 2 THEN Maths
    IF choice = 3 THEN Chem
  ENDWHILE
End program

Capital (start of procedure capital)
    answer is a local variable OF TYPE String
    score is a local variable OF TYPE Integer
    DISPLAY "What is the capital city of England?"
    ACCEPT answer
    IF answer = “London” THEN score:= score + 1
    DISPLAY "What is the capital city of France?"
    ACCEPT answer
    IF answer = “Paris” THEN score:= score + 1
    DISPLAY "What is the capital city of Japan?"
    ACCEPT answer
    IF answer = “Tokyo” THEN score:= score + 1
    DISPLAY "your score is", score
ENDPROCEDURE

Maths ( start of procedure maths)
    Use local variables number, score OF TYPE Integer
    DISPLAY "What is the square of 16?"
    ACCEPT number
    IF number = 256 THEN score:= score + 1
    DISPLAY "What is the square root of 81?"
    ACCEPT number
    IF number = 9 THEN score:= score + 1
    DISPLAY "What is the denary value of the hexadecimal number 21?
    ACCEPT number
```

```
    IF number = 33 THEN score:= score + 1
    DISPLAY "your score is ", score
ENDPROCEDURE
Chem ( start of procedure Chem)
    symbol is a local variable OF TYPE string
    score is a local variable OF TYPE integer
    DISPLAY "What is the chemical symbol for sodium?"
    ACCEPT symbol
    IF symbol = "Na" THEN score:= score + 1
    DISPLAY "What is the chemical symbol for chlorine?"
    ACCEPT symbol
    IF symbol = "Cl" THEN score:= score + 1
    DISPLAY "What is the chemical symbol for lead?"
    ACCEPT symbol
    IF symbol = "Pb" THEN score:= score + 1
    DISPLAY "your score is ", score
ENDPROCEDURE
```

In the simple example above, it is clear that the main procedure is calling the three lower level procedures. These do not then call other procedures, although to solve more complex problems, further calls to other levels are likely.

At the lowest level in this problem-solving technique a procedure, function or routine is written to solve a single problem. This code can then be used again whenever the same problem arises. This is clearly a great advantage because it means that code does not have to be repeatedly rewritten. In fact, once a reliable solution has been found, a library of such solutions can be kept for use whenever they are needed. This technique can be expanded further and reliable routines established which can be used whenever similar algorithms are required.

Exercise 4.1

The variable score occurs in each procedure of the Quiz program but is declared locally. What would have happened if score had been declared globally?

Using Parameters

The term parameter is used to describe variables which are used in a procedure or function to accept values passed to the procedure or function with the procedure call. The technical term for the value passed is an argument or actual parameter. The variables used within the procedure or function are called formal parameters. This complicated definition is best illustrated by two practical examples.

First Example

The first example describes a procedure that will output a character a number of times. The character and the number of times displayed will be chosen by the user before the procedure is called. Thus each time the procedure is called the instructions will be operating on data already chosen by the user. In order for the procedure instructions to be able to access this data, the procedure will need two parameters, one for the character and one for the number of times the character is to be displayed.

The pseudo code syntax for using parameters will be to put them in brackets with their data type written next to them. In this example we will use design as the procedure name and char and number as the two parameters. Thus the syntax for defining the procedure will be: -

PROCEDURE design (char OF TYPE Character, number OF TYPE Integer)

The variables char and number are called formal parameters, they are the ones defined and used within the procedure.

```
PROGRAM Pattern
Main Program
    choice, chosen_character OF TYPE Character
    number_of_times OF TYPE Integer
    choice:= “Y”
    WHILE choice = “Y”
        DISPLAY " What character would you like output to the screen?"
        ACCEPT chosen_character
        DISPLAY " How many times do you wish this to be sent?"
        ACCEPT number_of_times
```

```
        design (chosen_character, number_of_times)
        DISPLAY "Do you want another character? Type Y if you do"
        ACCEPT choice
    END WHILE
end program

PROCEDURE design (char OF TYPE character, number OF TYPE integer)
    index OF TYPE integer
    FOR index:= 1 TO number
        DISPLAY char,
    ENDFOR
ENDPROCEDURE
```

Note that the variables chosen_character and number_of_times are called actual parameters or arguments. These are the ones which are defined and used within the main program and contain the values which are passed to the procedure.

Definition: parameter

Information passed to a subroutine, procedure or function. The definition of the procedure is written using formal parameters to denote data items that will be provided when the subroutine is called, and the call of the procedure includes corresponding actual parameters. See also parameter passing.

Definition: argument

A value or address passed to a procedure or function at the time of call. Thus in the BASIC statement Y=SQR(X), X is the argument of the SQR (square root) function. Arguments are sometimes referred to as actual parameters.

Definition: actual parameter

Information passed to a subprogram at the call. See also parameter, argument.

Study Note

There are a variety of techniques for the declaration of parameters. The pseudo code

representation used here will need to be related to your chosen language.

Second Example

The second example will be a procedure called addup which uses two parameters, and then shows how it might be used in a program.

```
PROCEDURE addup(a, b OF TYPE Integer)
    answer OF TYPE Integer
    answer:= a + b
    DISPLAY"The sum is ", answer
ENDPROCEDURE
```

Whenever this procedure is needed, it is called by writing: addup(identifier1, identifier2). The identifiers are supplied by an earlier part of the program. The procedure would be used as shown below.

```
main program
    Use variables number1, number2 OF TYPE Integer
    DISPLAY "Enter two numbers "
    ACCEPT number1, number2
    addup(number1, number2)
end program
```

Both procedures used in the two examples have received data from the calling program and processed the data within the procedure. The actual arguments in the calling program have not been changed by the procedure.

These procedures are examples of passing parameters by value, a one-way communication of data from the calling program to the procedure. An alternative, passing by reference, which involves two-way communication between the calling program and the procedure, will be addressed later in the chapter.

Exercise 4.2

Explain the difference in use between actual and formal parameters. Illustrate your answer by reference to the pseudo code solution to Example 2 above.

Functions

The main difference between a function and procedure is that a function returns a value to the calling program. Most programming languages contain a library of functions, e.g. to generate a random number, to calculate a variety of mathematical functions such as square root, to find and return values from lists. For example: -

```
ACCEPT X
DISPLAY random(X)
```

This short piece of code accepts the argument you input for X, generates a random number in the range 1 to X and returns the answer to be displayed on the screen.

Note that functions can be used on the right hand side of an assignment, for example: -

```
random_number:= random(X)
```

In our pseudo code we will write the functions using the same construct as that for procedures but using the term ENDFUNCTION to show the finish of the code and the word RETURN to indicate the identifier sending the value back to the calling place. Note that the call may come from any other procedure or function.

We shall now look again at the simple mathematics example program used in Chapter 2 and use it to illustrate the concept of parameters to functions and how a value is returned from a function.

The program steps required were: -

1. Present a menu of choices to the user; add, subtract, multiply, divide.
2. Repeat ‘accept the choice and do the calculation’ until end of program is requested.

We will use two procedures in the program, one to display the opening message instructions to the user and the second to accept the choice and do the calculation. However, this time we will make the calculation part of the procedure a function. The

pseudo code for the main program and opening message procedure is:

```
main program
    opening_message
    screen_menu
end program

PROCEDURE opening message
    DISPLAY "This is a simple mathematics program."
    DISPLAY " It will offer you a choice of calculation",
    DISPLAY " to be performed on any two numbers you input."
ENDPROCEDURE
```

The pseudo code for the menu procedure is: -

```
PROCEDURE screen_menu
    use variables: number1, number2, result OF TYPE Integer
    DISPLAY " Enter your two numbers"
    ACCEPT number1, number2
    DISPLAY "Now choose your calculation type"
    DISPLAY "When you are finished type the number 5"
    REPEAT
        DISPLAY "1: addition"
        DISPLAY "2: subtraction"
        DISPLAY "3: multiplication"
        DISPLAY "4: division"
        DISPLAY "5: endprogram"
        ACCEPT choice
        CASE choice OF
            CASE 1: result = addup(number1,number2)
            CASE 2: result = subtract(number1,number2)
            CASE 3: result = multiply(number1,number2)
            CASE 4: result = divide(number1,number2)
```

```
        ENDCASE
        DISPLAY result
    UNTIL choice = 5
ENDPROCEDURE
```

The program calls a function called addup and passes two items of data to it as arguments. The data stored in the variables number1 and number2 will be used in the function to calculate an answer. This will be returned and stored in the identifier result. The pseudo code for the function is: -

```
INTEGER FUNCTION addup(a,b OF TYPE integer)
    answer:= a+b
    RETURN answer
ENDFUNCTION
```

Note that the data type of the variable answer has not been defined. This is because the function itself has been defined as Integer. This relates to the data type of the data that is returned by the function. The function uses local variables a and b, of type integer, to calculate and return an answer which is also an integer.

Exercise 4.3

Write the pseudo code for the other three functions, subtract, multiply and divide. Do not allow the user to divide by zero and for simplicity, let the result of the divide be an integer.

Summary of procedures and functions

There are a number of important reasons for using procedures/ functions: -

- the code can be built to reflect a structured design with procedures echoing the design at each level of the top down process;
- tasks that are needed repeatedly within a program can be coded once and then called when needed;
- a smaller size of procedures makes them easier to test and debug, leading to greater overall reliability;

- variables can be confined to the procedure in which they are used, thus eliminating unexpected side effects in other parts of the program.

Arrays

Arrays are simply a sequence or list of data items, all of which are the same data type. Each data item can be referenced by its position in the sequence. The normal programming method of referencing an array is to use the array name, followed by a subscript containing the element number. In mathematics the third element of an array called marks would be referenced as marks3 but in programming it would be referenced as marks(3) or marks[3] depending on the language.
We will use the square brackets as our convention.

The array is stored in the computer in adjacent memory locations and each memory location would have an identifier composed of the array name followed by the appropriate subscript. Arrays can be one or two dimensional (or more). Some programming languages use simple lists instead of arrays. The reference used will be different but the principles are the same.

Definition: Arrays

An ordered collection of a number of elements of the same type, the number being fixed unless the array is flexible. The element of one array may be of type integer, those of another array may be of type real, while the elements of a third array may be of type character string (if the programming language recognizes compound types). Each element has a unique list of index values that determine its position in the ordered collection. Each index is of a discrete type. The number of dimensions in the ordering is fixed.

A one-dimensional array, or vector, consists of a list of elements distinguished by a single index. If v is a one dimensional array and i is the index value, then vi refers to the ith element of v. If the index ranges from L through U then the value L is called the lower bound of v and U is the upper bound. Usually in mathematics and often in mathematical computing the index type is taken as integer and the lower bound is taken as one.

Definition: index

A list of values of some particular data item contained in a record, enabling it to be retrieved more rapidly than by simple serial search. For example, a subscript is a value, usually integral, that selects a particular element of an array.

Definition: subscript

A means of referring to particular elements in an ordered collection of elements. For example, if R denotes such a collection of names then the ith name in the collection may be referenced by Ri (i.e. R subscript i). This printed form is the origin of the term but it is also used when the "subscript" is written on the same line, usually in parentheses or brackets: R (i) or R [i].

One Dimensional Arrays

This type of array is a sequence of data items where each item or element can be referenced by the position it occupies in the sequence. The array elements can be simple or complex data types but they must all be the same, i.e. homogeneous.

Example 1

A list of numbers can be defined as an array, for example the examination marks for a student. An array named marks contains 23, 34, 45, 66 in that order. Each element can now be identified as: -

23 is marks[1] , 34 is marks[2], 45 is marks[3] and 66 is marks[4].

Example 2

Name is defined as an array of characters such that if the word James is stored in the array called name, then "J" is name[1], "a" is name[2], "m" is name[3], "e" is name[4] and "s" is name [5].

Exercise 4.4

A deck of 52 cards (no joker) is sorted into the suit sequence hearts, clubs, diamonds and spades, with each suit in ascending order of value, from 2 to 10, then Jack, Queen, King and Ace. Defining this structure as an array called cards, answer the following questions:

a) Name the cards identified by cards[3], cards[51], cards[27].

b) What are the identifiers for the queen of hearts, the jack of diamonds and the ace of spades?

The following pseudo code is for a program which will read in a sequence of 10 numbers and then print them out in reverse order.

```
PROGRAM reverse
    Use variables: numbers[10] : ARRAY OF TYPE Integer
                   count, n OF TYPE Integer
    FOR (n:=1, n=10, +1)
        Display " Enter a number"
        ACCEPT numbers[n]
    ENDFOR
    DISPLAY " The numbers you entered in reverse order are"
    FOR (count:=10, count=1, -1)
        DISPLAY numbers[count]
    ENDFOR
endprogram
```

One-dimensional arrays can be used with other one-dimensional arrays where the data in each array is of a different type, for example, the marks for an examination for students in a class. We could use a one-dimensional array called names where the data items are of type character and another one-dimensional array called marks which contains data of type integer. In this case the subscript in each array could always refer to the data for the same student, as long as this method was also used on data entry. It's simply a matter of making sure that both lists are in the same order. If, on data entry, the name of the student is added to the next element in the name array and the corresponding mark is entered in the next element of the marks array, the two lists will correspond. If the marks list is then sorted into ascending marks order, resulting in the data items moving place in the marks array then the elements of the name list must also be changed accordingly. For example: -

subscript	names array	marks array
1	Peters J	55

2	Ratcliffe N	43
3	Smith J	67
4	Sutcliffe B	34
5	Yates V	78

Figure 4.2: Two arrays showing name of student and marks

Example program

We will write the pseudo code to input the name of the student and examination mark for each student in a class of ten. At this stage, the marks will just be displayed. Later in this chapter you will be sorting the marks into descending order and displaying a list of student names and marks starting with the top of the class. You will also, later in this chapter, learn how to display the marks for a particular student by looking up the name in the names array and displaying the matching element in the marks array.

```
PROGRAM enter and display examination results
    Use variables:
    marks[10] : ARRAY OF TYPE Integer
    names[10] : ARRAY OF TYPE String
    count, n OF TYPE Integer
    FOR (n:=1, n=10, +1)
        DISPLAY " Enter the student's name"
        ACCEPT names[n]
        DISPLAY "Enter the mark for ", names[n]
        ACCEPT marks[n]
    ENDFOR
    DISPLAY " The examination results are"
    FOR (count:=1, count =10, +1)
        DISPLAY names[count], marks[count]
    ENDFOR
endprogram
```

If an array of marks was defined as a maximum of 100, then this could be changed to a procedure to enter and display the marks. The actual number of elements in the array would be determined during the data entry part of the procedure, e.g. the user could enter

the number of marks to follow or the program could count them until a terminating value was entered. This parameter would be updated by the procedure and passed back to the main program where it could be used in every call statement to a procedure which accesses the array.

This is an example of parameter passing by reference rather than by value. This is like a two way communication between the calling program and procedure in that the data can be changed by the procedure and then accessed by the calling program. You will be using reference parameters in Exercise 4.5.

Definition: flexible array
An array whose lower and/or upper bounds are not fixed and may vary according to the values assigned to it. See also string.

Definition: string
Any one-dimensional array of characters.

Exercise 4.5
Amend the pseudo code above to provide an algorithm for a procedure to enter each student's name and mark for a class of any number up to a maximum of 100. Provide the statement which will call the procedure and remember to validate the number of marks to be entered into the array.

Two Dimensional Arrays
Two dimensional arrays have two subscripts to identify an element of the array. They can be thought of as the rows and columns in a table. This is rather similar to the table produced in Figure 4.6 but you couldn't use that table as a two dimensional array because the names data elements were of type String and the marks were of type Integer.

All the elements of a two dimensional array must be of the same data type. To illustrate two dimensional arrays, think of an array of words such as a list of first names. Since a word is an array of characters, then an array of first names is in fact a two dimensional array and in some languages has to be declared as such. The convention in this case is to

use two subscripts. We shall show this by a practical example. The array is called name_list and consists of 5 names of no more than 15 characters: james, janet, alexander, lee, zacharia.

If we place them in a table (as in Figure 4.3) you can clearly see the relationship between the two dimensional array and the rows and columns. The row number would refer to the one dimensional array for one name where each element is a character in the name.

	COLUMNS										
ROWS		1	2	3	4	5	6	7	8	9	10
	1	j	a	m	e	s					
	2	j	a	n	e	t					
	3	a	l	e	x	a	n	d	e	r	
	4 5	l z	e a	e c	h	a	r	i	a		

Figure 4.3: Two dimensional array of names and characters

The letter “m” in the name james can be precisely located as name_list[1, 3]. Similarly the letter “z” in name zacharia is name_list[5, 1]. Note that the “s” in the name james would be referenced as name_list[1, 5].

It is important to know how the two dimensional array will be referenced in terms of whether the row or column subscripts come first and you must identify this for the programming language you will be using. We will reference the row number first so name_list[1, 5] will always mean row 1, column 5 in the array.

A process to sort these names into alphabetical order would have to compare each name, character by character, to determine the appropriate ranking. It would not be sufficient simply to compare each first letter since these might be the same, as in the example where James and Janet can only be ordered on a comparison test when the third letters are compared, i.e. name_list[1, 3] < name_list[2, 3].

Definition: matrix

A two-dimensional array. In computing, matrices are usually considered to be special cases of n-dimensional arrays, expressed as arrays with two indices. The notation for arrays is determined by the programming language. The two dimensions of a matrix are known as its rows and columns; a matrix with m rows and n columns is said to be an m n matrix.

Exercise 4.6

ROWS	COLUMNS					
		1	**2**	**3**	**4**	**5**
	1	1	2	3	4	5
	2	2	4	6	8	10
	3	3	6	9	1	15
	4	4	8	12	16	20

The multiplication table above shows the five times table for the numbers 1 to 4. Write the pseudo code to calculate the 12 times table for the numbers 1 to 15. HINT: Use a two dimensional array with rows 1 to 15 and columns 1 to 12. Use the fact that the row and column (subscript) values can be used in the calculation. You will be using nested FOR ... ENDFOR loops.

Tables: Arrays of Records

A table of records can be considered as a two dimensional array in which the rows represent the records, and the columns the fields. Note that all data items of an array have to be of the same type. In the Figure 4.4 example, all data items are of type character so we could treat the contents of the table as a two-dimensional array called records.

FIELD NAMES				
RECORDS	**First name**	**Last name**	**Grade**	**Job type**
	John	Chang	A1	Engineer
	David	Smith	A5	Computing
	Peter	Lee	A4	Admin
	Wiseman	Mponda	A4	Writer

Figure 4.4: Records and Field names in a two dimensional table

However, we could define the record structure by identifying the fields within the record and then the array will reduce to a one dimensional array where the subscript relates to the record number. The advantage of this method is that all the fields do not have to be the same data type.

Exercise 4.7

With reference to the data in the table, Figure 4.4, answer the following: -

a) What data will be retrieved from elements: records[1, 1], records[4, 4], records[3, 2]?

b) What are the identifiers for David Smith's job type, Peter Lee's grade of post and David Smith's first name?

c) If another record was added for Elizabeth Hardy who works as a manager at a7 grade, what identifiers would be used to store the data?

Definition: record

A data structure in which there are a number of named components, called fields, not necessarily of the same type. It may have variants in which some of the components, known as variant fields, are absent; the particular variant for a given value would be distinguished by a discriminate or tag field. The record is widely recognized as one of the fundamental ways of aggregating data (another being the array) and many programming languages offer direct support for data objects that take the form of records (see structured variable). Such languages permit operations upon an entire record object as well as upon individual components.

Definition: field

An item of data consisting of a number of characters, bytes, words, or codes that are treated together, e.g. to form a number, a name, or an address. A number of fields make a record and the fields may be fixed in length or variable.

Pseudo code for the Record Data Structure

The record will be given a name and each field of the record will be declared with a name and data type. The one dimensional array will then be defined by providing an identifier, identifying the number of elements and declaring that the array is based on the record

structure. The fields can be referred to individually as in the following example. To declare a record called car with fields, make, size, and colour.

RECORD car
HAS FIELDS make : String
size : Integer
colour : String

To declare an array, which contains 20 of these records: -

types_of_car [20] ARRAY OF TYPE car

To refer to each of the fields individually in pseudo code we would type for example: -

types_of_car[3].make to refer to the make field in the 3rd record
types_of_car[1].size to refer to the size field in the 1st record
types_of_car[9].colour to refer to the colour field in the 9th record

Exercise 4.8
Produce the pseudo code to define the record data structure and array for the personnel data concerning jobs and grades shown in Figure 4.4.

Figure 4.2 displayed the names and marks for a class of students and this data were previously manipulated using two one-dimensional arrays. The record data structure could now be applied to this problem.

Exercise 4.9
Produce the pseudo code to define the record data structure and array for the student examination marks shown in Figure 4.2: Assume a class of 25.

Now that you know how to define arrays of records it is time to incorporate them into programs. We will use the record structure you defined for the answer to Exercise 4.9.

Example program using arrays of records

The problem: A procedure is required to output the name of the student with the highest mark from a series of names and marks entered at the keyboard and stored in an array of records. There are currently 12 members of the class and the class size will never be more than 100. The procedure is called by the statement highest (results, 12). The algorithm for finding the highest mark is: -

1. Store the subscript of the highest mark in an identifier highest.
2. Assume initially that the first mark is the highest and every time you find one higher than this, make that one the highest by saving it's subscript in the identifier highest.
3. When every element in the array has been checked then the elements student[highest].name and student[highest].mark can be displayed.

```
PROCEDURE highest(classlist, number_of_students)
    RECORD student
    HAS FIELDS name: String
               mark: Integer
    classlist[100] ARRAY OF TYPE student
    n, number_of_students, highest OF TYPE Integer
    FOR (n:=1, n=number_of_students, +1)
        DISPLAY " enter student name"
        ACCEPT student[n].name
        DISPLAY " enter student mark"
        ACCEPT student[n].mark
    ENDFOR
    -- find highest
    highest :=1
    FOR (n:=2, n=number_of_students, +1)
        IF student[n].mark > student[highest].mark
            THEN highest:=n
        ENDIF
    ENDFOR
    -- display highest mark
```

DISPLAY" The student with the highest marks is " student[highest].name
DISPLAY" The mark was" student[highest].mark
END PROGRAM

Exercise 4.10

This procedure could be made more useful if the number_of_students could be determined in the procedure at the same time as the array data was entered. Assuming a maximum of 100 in a class, what changes would you make to the above pseudo code to add the following: -

Allow the user to input the number in the class before entering the list of names and marks. Validate this number it must be between 1 and 100 inclusive. If the user inputs a number outside the range, provide an error message and ask for another number. Provide an example of the calling statement to this procedure.

Arrays: Sorting and Searching

The techniques of external sorting will not be considered in this workbook. The following section deals with internal sorts, i.e. performed in the memory of the computer.

There are many different algorithms for sorts and this section only gives you three examples of the different types. The algorithms for sorting will work just as well for arrays of tables as for one-dimensional arrays. The different sorting strategies will be illustrated by the use of lists for presentational simplicity.

The following sorts will be looked at: -

- selection sorts;
- selection sort using two arrays;
- straight selection sort;
- straight insertion sort;
- exchange selection sort often called a bubble sort.

Definition: sorting

Rearranging information into ascending or descending order by means of sort keys. Sorting may be useful in three ways: to identify and count all items with the same

identification, to compare two files, and to assist in searching, as used in a dictionary. An internal sorting method keeps the information within the computer's high speed RAM; an external sorting method uses backing store. There are a wide variety of methods.

Definition: sort key

The information, associated with a record of information, which is to be compared in a sorting process. It follows that the sort keys must be capable of being ordered, i.e. two keys k1 and k2 are such that k1 < k2, k1 = k2 and k1 > k2.

Selection Sorts

Example 1: A Selection Sort Using Two Arrays

This sort uses two arrays, the original one and an extra one which is originally empty but is then gradually filled with data items in the required order. The technique is outlined below: -

- Look through the original array to find the lowest element (for a sort in ascending order).
- This element is written to the empty array in position one and the element in the data array marked by overwriting the key with a rogue value, i.e. one which is clearly identified as not belonging to the actual data.
- The search is repeated finding the next lowest key (ignoring rogue values) and writing this in position two in the second array, and overwriting as before. Each of these searches, where the whole array is looked at, is called a pass.
- The process continues until the sort is complete.

Note that this would be a very inefficient sort for large arrays of data. Figure 4.5 shows the contents of the two arrays after each pass during the search. The initial state of the data array is 205, 310, 45 and the extra array is empty.

Starting state		**After pass 1**		**After pass 2**		**After pass3**	
Data array	**Extra array**	**Data array**	**Extra array**	**Data array**	**Extra array**	**Data array**	**Extra array**
205		205	**45**	**999**	45	999	45
310		310		310	**205**	**999**	205
45		**999**		999		999	**310**

Figure 4.5: Table showing the contents of elements of the array after each pass

Definition: pass

A single scan through a body of data, for example by a compiler reading the program text or a statistical package reading its data.

Definition: rogue value

A value added at the end of a table and that can be recognized as a termination signal by a table lookup program.

Example 2: Straight Selection Sort

The following description outlines an algorithm for a selection sort using an exchange of variables which avoids the use of the extra array needed by the previous method. In this straight selection sort: -

- The array is scanned to find the location of the smallest element. This element is then exchanged with the first member of the array. This is called the first pass; the whole data has been looked at to find the smallest item.
- The array is scanned again, starting at the second member, the location of the smallest remaining member is determined, and the element at this location exchanged for the second member of the array. This is called the second pass.
- The process continues until all array elements have been scanned and placed in their correct position.

This will involve using nested loops. The outer loop will control the pass, the inner loop to compare the elements from that point to find the lowest element.

Pseudo code solution for a straight selection sort

This algorithm is to sort an array of size n using a basic selection technique.

```
VARIABLES
numbers[n]: ARRAY OF TYPE Integer
count, pass, lowest,temp OF TYPE Integer
FOR (pass:= 1, pass = n - 1, +1)
    count:= pass + 1
```

```
    lowest:=pass
    FOR (count:= pass+1, count = n, +1)
        IF numbers[count] < numbers[lowest]
            THEN lowest:= count
    ENDFOR
    temp:= numbers[pass]
    numbers[pass] := numbers[lowest]
    numbers[lowest] := temp
ENDFOR
```

Figure 4.6 shows the contents of the elements of the array after each pass and Figure 4.7 shows the contents of the variables pass, count, lowest and temp during the 4 passes.

subscript	array	Original		Array contents after	
		1st pass	2nd pass	3rd pass	4th pass
1	205	**45**	45	45	45
2	310	310	**67**	67	67
3	67	67	**310**	**205**	205
4	45	**205**	205	**310**	**263**
5	263	263	263	263	**310**

Figure 4.6: Table showing the contents of each array element after each pass of a straight selection sort

Notice how the data item 310 was moved a number of times during this sort before it ended up in the correct place. This sort is quite useful if the data is already in a reasonable sequence.

Variables	Variable contents during			
	1st pass	2nd pass	3rd pass	4th pass
pass	1	2	3	4
count	2, 3, 4, 5	3, 4, 5	4, 5	5
lowest	1, 3, 4	2, 3	3, 4	4, 5
temp	205	310	310	310

Figure 4.7: Table showing the contents of the variables used during each pass of a

straight selection sort

Definition: straight selection sort

A sorting algorithm based upon finding successively the record with the largest sort key and putting it in the correct position, then the record with the next largest key etc.

Exercise 4.11

a. Write the pseudo code for a program which accepts 6 numbers from the keyboard, stores them in an array, sorts them using a straight selection sort and then displays them in ascending order.

b. Using test data of 56, 34, 78, 45, 89, 23 provide a table showing the contents of the elements of the array as shown above in table 4.6.

Straight Insertion Sort

This is an effective sort when sorting small arrays but rather slow when sorting larger arrays. The algorithm is: -

- On the first pass, the first two elements of the array are compared and arranged in order.
- On the second pass the third element is compared with the first two and inserted into the correct position. This involves moving the third element into the correct position and shuffling all the other elements along.
- The process is repeated, on each pass comparing the current element with the previous ones to see where its position should be, until every element in the list has been placed in its correct position.

Before looking at the pseudo code, study the table Figure 4.8 which shows the contents of the elements of the array after each pass. The original list of numbers is 21, 16, 35, 47, 19, and 12.

Initial array	**1st pass(pass=2)**	**2nd pass(pass=3)**	**3rd pass(pass=4)**	**4th pass(pass=5)**	**5th pass(pass=6)**
21	16	16	16	16	12
16	21	21	21	19	16
35	35	35	35	21	19
47	47	47	47	35	21
19	19	19	19	47	35

12	12	12	12	12	47

Figure 4.8: Table showing the contents of the array elements after each pass of a straight insertion sort

Notice that in the 4th pass, 19 is moved from the 5th element to the 2nd element and the old elements 2, 3 and 4 are moved along the array by one subscript position. Thus the 5th element has been inserted in its correct position in the array.

The following pseudo code is the algorithm for the straight insertion process shown above, assuming the numbers are stored in the array show.

```
VARIABLES
    show[6]: ARRAY OF TYPE Integer
    position, pass, copy OF TYPE Integer
    FOR (pass = 2, pass = 6, +1)
        copy := show[pass]
        position := pass
        WHILE position > 0 AND show[position - 1] > copy
            show[position] := show[position - 1]
            position := position - 1
        ENDWHILE
        show[position] := copy
    ENDFOR
```

Definition: straight insertion sort (sifting technique, sinking technique)
A sorting algorithm that looks at each sort key in turn, and on the basis of this places the record corresponding to the sort key correctly with respect to previous sort keys.

Exercise 4.12
Using test data of 56, 34, 78, 45, 89, 23 provide a table showing the elements of the array during the processing of a straight insertion sort as shown in Figure 4.8.

Exchange Selection Sort
This is called a bubble sort because the largest element rises to the top of the array (i.e.

the element with the largest subscript) in the first pass, and then the next largest bubbles up to the next position in the second pass, and so on. Of course the array could be sorted in descending order, when the smallest element would be the one bubbled to the top.

The algorithm is simple and effective, particularly when none of the elements are far from their final position and there are only 20 30 elements. The algorithm is: -

- On the first pass, the first two members of the array are compared and exchanged if necessary. The process is repeated with the second and third elements and then the third and the fourth and so on until the largest element arrives at the top, i.e. in the element with the highest subscript.
- Now that one element is in the correct position the process is repeated until the next largest is in the next to the end position. However, it is not necessary to include the last element as that is already in the correct position. Thus during each successive pass, one less element is considered than the previous pass.
- This is repeated until every element has been sorted. This will happen when the complete cycle of passes has ended, or as soon as no more exchanges have taken place. This obviously means that a record has to be kept of whether any exchanges have taken place during a pass (if not, then all elements must be in the correct order).

Assume that the original list is 3, 1, 2, 5, 4.

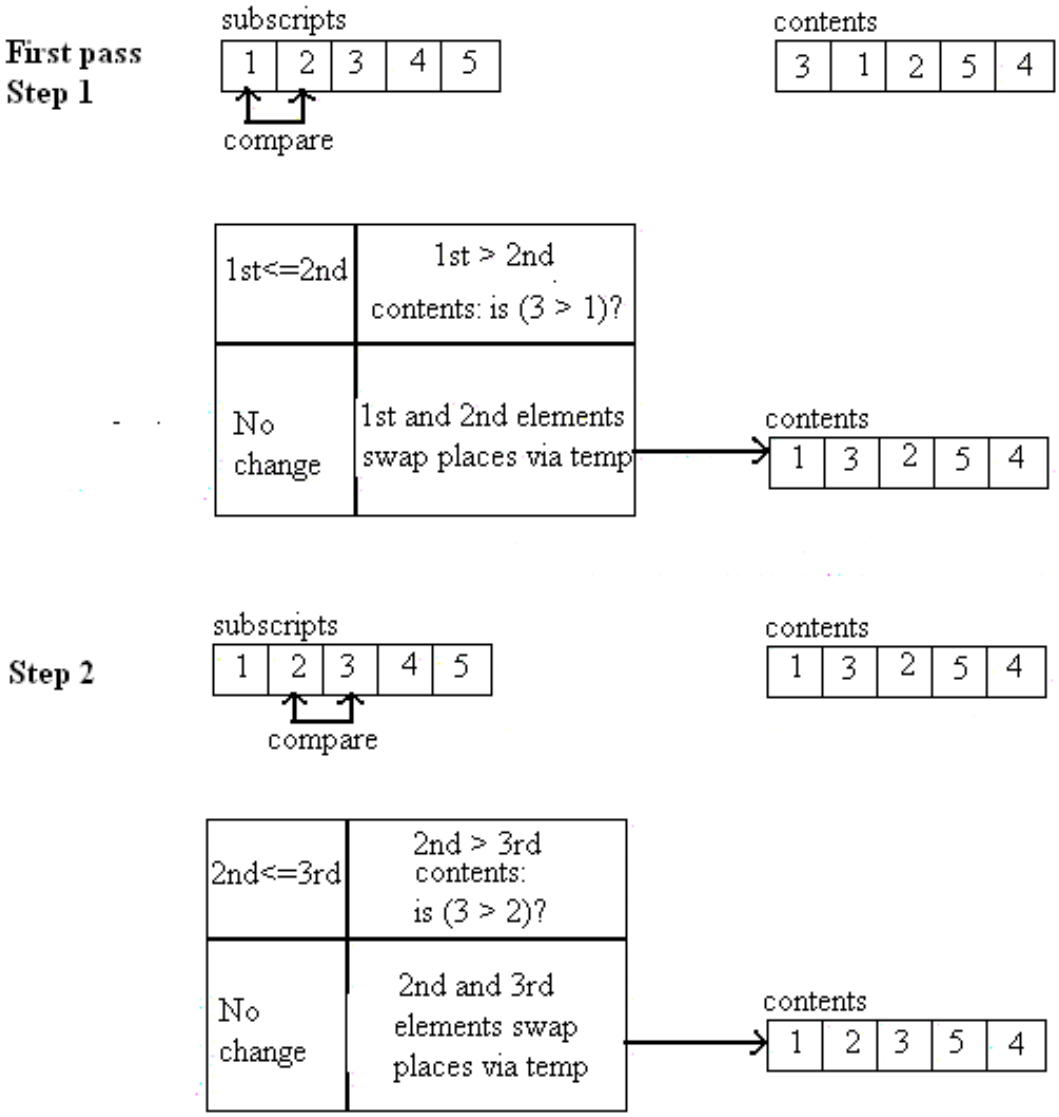


Figure 4.9: Diagram showing the first two steps in an exchange selection sort

The pseudo code for a bubble sort of an array of 10 integers is: -

```
main program
    Use local variables
        example [10]: ARRAY OF TYPE Integer
        exchange_count, pass, temp OF TYPE Integer
    pass:=1
    REPEAT
        exchange_count:= 0
```

```
        FOR (n=1, n = 10 - pass, +1)
            IF example[n] > example[n+1]
                THEN temp:= example[n]
                example[n] := example[n+1]
                example[n+1] := temp
                exchange_count := exchange_count+1
            ENDIF
        ENDFOR
    pass:= pass+1
    UNTIL exchange_count= 0
endprogram
```

Definition: bubble sort (exchange selection)

A form of sorting by exchanging that simply interchanges pairs of elements that are out of order in a sequence of passes through the file, until no such pairs exist.

This method is not competitive with straight insertion.

To illustrate the logic behind this pseudo code, consider a sequence of numbers 41, 23, 42, 32, 67, 53, 46, 99, 81, 73. Figure 4.10 shows the contents of the elements of the array at the end of each pass and how the variables change during the loop.

Array element	**Initial state**	**After 1st pass**	**After 2nd pass**	**After 3rd pass**
1	41	23	23	23
2	23	41	32	32
3	42	32	41	41
4	32	42	42	42
5	67	53	46	46
6	53	46	53	53
7	46	67	67	67
8	99	81	73	73
9	81	73	81	81
10	73	99	99	99
Variables used during loop				
pass	1	2	3	4
n		1,2,3,4,5,6,7,8,9	1,2,3,4,5,6,7,8	1,2,3,4,5,6,7
temp		41, 42, 67, 99	41, 53, 81	

exchange_count	1	0,1,2,3,4,5,6	0, 1, 2, 3	0
Thus the program will end after the 3rd pass as exchange_count = 0				

Figure 4.10: Table showing contents of elements and variables during each pass

Exercise 4.13

Using test data of 56, 34, 23, 45, 89, 78 provide a table showing the elements of the array during the processing of an exchange selection (bubble) sort as shown above in Figure 4.10.

Searching

Finding one particular element in an array is simply a matter of looking through the array, element by element, until the required key is found. This can be done sequentially, i.e. starting from the first element and looking for a matching key until one is found. For example: - IF key_to_be_found = array[n] THEN found := TRUE

The identifier found is of type Boolean. This means that found can only have two values either 0 or 1, where 0 is the same as FALSE and 1 is the same as TRUE.

They are used by programmers as switches to indicate one of two states. This can be useful in the logic of the program as it allows the programmer to record the state of things in certain situations; as in this case where it is used to record the fact that the item has been found. The identifier used can then be tested in other parts of the procedure or may be returned through a function. The pseudo code for this type of search is shown below: -

```
-- note that key_to_be_found and key_array would contain data at
-- this point of the program. The key_to_be_found data could have
-- been input over the keyboard
Use local variables
    key_array: ARRAY [10] OF TYPE String
    key_to_be _found OF TYPE String
    count, n OF TYPE Integer
    found OF TYPE Boolean
begin sort procedure
    Count := 1
```

```
found := FALSE
REPEAT
    IF key-to-be-found = key_array[count]
        THEN found := TRUE
        ELSE count :=count+1
UNTIL found = TRUE OR count >10
-- the program would then continue
-- taking appropriate actions with the item found - element count
-- or a "not found" result
```

Since it is important to also determine if the particular key is present, it may be necessary to search the entire array. This method is clearly inefficient (imagine the number of names in a telephone book) although for small searches this may not matter.

Searching a sorted list

A more efficient method is to start by sorting the array on the key to be found. A test can then be included to stop the search when the key is matched or when the key reached is greater than the key to be found. This would involve: -

- Processing the array, one element at a time, in a loop.
- Comparing the key_to_be_found with each successive element in the array stopping the loop when a match is found, as described in the previous example.
- In addition, checking whether the current element in the array is greater than the key_to_be_found. If this is the case then it means that you have passed the position where key_to_be_found should be and thus can conclude that the key_to_be_found is not there. For example: - IF key_to_be_found > array[n] THEN ....

For example, consider the list of names used earlier in the chapter in Figure 4.2 (reproduced here for your convenience).

subscript	names array	marks array
1	Peters J	55
2	Radcliff N	43
3	Smith J	67
4	Sutcliffe B	34

5	Yates V	78

Copy of Figure 4.2 Two arrays showing name of student and marks

The names are in sorted order so can be used to illustrate the search method described above. If the name “Smith J” was entered over the keyboard then key_to_be_found:= “Smith J”. “Smith J” is greater than “Ratcliffe N” but is less than “Sutcliffe B”. If you were doing the search, you would stop as soon as you found an entry higher than the required one. This is exactly how the search algorithm works. As the list is supposed to be in alphabetical order (i.e. sorted), then it is safe to assume that “Smith J” is not in the list.

This is the pseudo code algorithm for the search of the sorted array of 10 key names.

```
    key_array: ARRAY [10] OF TYPE String
    Use variables key_to_be _found OF TYPE String
                count, n OF TYPE Integer
                found, missing OF TYPE Boolean
-- note that key_to_be_found and key_array would contain data at
-- this point of the program. The key_to_be_found data could have
-- been input over the keyboard
begin sort procedure
    count:=1
    found:= FALSE
    missing:= FALSE
    REPEAT
        IF key-to-be-found = key_array[count]
            THEN found := TRUE
            ELSE IF key_to_be_found < key_array[count]
                THEN missing:= TRUE
        ENDIF
        count :=count+1
    UNTIL found = TRUE OR count >10 OR missing = TRUE
-- the program would then continue
```

-- taking appropriate actions with the item found

-- or a "not found" result

The Binary Search

This method can substantially reduce search times when dealing with larger arrays. There is more processing per comparison, since a calculation is required. However, the number of comparisons is less. The technique is as follows: -

1. Determine the start and end subscript values for the array, say, start and end.
2. Find the centre of the array subscript = (start + end)/2.
3. Compare the key required with the key at this location.
4. If the key required is less than the key found, the key we are looking for is in the first part of the array. Otherwise, it is in the second half of the array.
5. This process, 1 to 4 above, can now be repeated with the appropriate section of the array, i.e. a new start or end subscript value based on the previous mid-point subscript and calculate a new mid-point. This process is continued until the key is found.
6. It is, of course, possible that the key may not be found at all and an allowance must be made for this eventuality.

Pseudo code for this Algorithm

Start is used as the lower boundary of the section currently being searched; end is used as the higher boundary of the section currently being searched. At the beginning of the search, these are the subscripts of the first and last elements of the array.

```
Use variables start, end, middle OF TYPE Integer
        found OF TYPE Boolean
        key_required OF TYPE String
        key_name (10): ARRAY OF String
start:= 1
end:= 10
found:= false
REPEAT
    middle:= (end+start)/2
    IF key_required = key_name[middle]
```

```
        THEN found := TRUE
        ELSE IF key required < key-name[middle]
            THEN end : = middle-1
            ELSE start := middle+1
    ENDIF
UNTIL found= TRUE OR start > end
```

Exercise 4.14

Amend the pseudo code to provide a function which will perform a binary search on a sorted array of variable length and return the subscript number of the found item in the array. Note that this means that you must pass the size of the array as one of the function arguments. Assume a maximum of 100 elements in the declared array.